PaceFlow: AI-Driven Pacing Co-Pilot

1. Brainstorm & Expansion

Product Overview

PaceFlow is a production-grade interactive pacing co-pilot designed for runners who find standard tracking tools insufficient for high-stakes, time-bound performance goals. Unlike static split calculators, PaceFlow adjusts dynamically to the environment and the athlete's biological state.

Strategic Foundation

- **Core Problem:** Traditional trackers rely on static pace splits or simple heart-rate warnings. They fail to account for 3D terrain (climbs/descents), muscle fatigue, wind resistance, heat index, and biomechanical form degradation, leading to premature energy depletion or missed targets (e.g., failing a sub-2 hour half-marathon).
- **Target Audience:** Intermediate and advanced runners engaged in systematic training to break through performance plateaus.
- **Unique Value Proposition:** An AI-driven co-pilot utilizing 3D elevation modeling, real-time biomechanics analysis, and predictive Monte Carlo simulations to deliver terrain-adjusted audio coaching and form correction in real-time.

Core MVP Features

1. **3D Elevation & Weather-Adjusted Pace Curve Generator:** Calculates the optimal pace for every meter based on slope and environmental conditions.
2. **Live Biomechanical Form Monitor:** Real-time analysis of cadence, ground contact time, and vertical oscillation with audio feedback.
3. **Monte Carlo Goal Simulator:** Pre-race engine that runs thousands of scenarios to predict finishing times based on varied physiological and environmental inputs.

4. **Fatigue Heat-Map Overlay:** A post-run visual analysis tool that maps form degradation onto the geographic route to inform training recalibration.

Modern Tech Stack

- **Frontend:** Flutter (Cross-platform iOS/Android and watchOS/Wear OS companions).
- **Backend:** Node.js, Express, TypeScript.
- **Data Layer:** PostgreSQL with Prisma ORM.
- **Integrations:** Garmin SDK, Apple HealthKit, Google Fit.
- **AI/Voice:** OpenAI API for real-time synthetic coaching and logic compilation.

2. MASTER PROJECT.md

PROJECT.md: PaceFlow Core Architecture

1. Project Directory Structure

paceflow/

├── apps/

│ ├── mobile/ # Flutter (iOS/Android)

│ └── wearable/ # Flutter/Native (watchOS/Wear OS)

├── backend/

│ ├── prisma/ # Schema and Migrations

│ ├── src/

│ │ ├── api/ # Express Routes

│ │ ├── services/ # Physics, Simulation, & Auth Logic

│   │   ├── middleware/   # Telemetry Validation

│   │   └── index.ts   # Entry Point

│   └── tests/   # Unit & Integration Tests

└── infrastructure/   # Docker & CI/CD Configs

2. Database Schema (PostgreSQL)

- **Users**: ID, Email, Weight, VO2Max, Threshold HR, CreatedAt.

- **TrainingPlans**: ID, UserID, TargetDate, GoalTime, GoalDistance.

- **Courses**: ID, Name, GPX_Data (JSON), TotalElevationGain, HeatIndexFactor.

- **RunSessions**: ID, UserID, CourseID, Date, TotalTime, AvgPace.

- **TelemetrySamples**: ID, SessionID, Timestamp (ms), Lat, Lon, Elevation, Cadence, GCT (Ground Contact Time), VerticalOscillation, HeartRate.

- **PredictiveSimulations**: ID, UserID, CourseID, ScenarioParams (JSON), ProbableFinishTime, ConfidenceInterval.

3. Core API Endpoints

Course & Pacing

- `POST /api/v1/courses/import`: Accepts GPX files and calculates slope coefficients.

- `GET /api/v1/pacing/curve`: Returns a terrain-adjusted pace curve for a specific course.

Telemetry & Sync

- `POST /api/v1/sync/wearable`: Ingests millisecond-level telemetry from Garmin/Apple Health.

- `GET /api/v1/sessions/:id/metrics`: Retrieves biomechanical form data for a specific run.

Analytical Engine

- `POST /api/v1/simulations/run`: Triggers the Monte Carlo engine for a selected course and goal.

- `GET /api/v1/analytics/fatigue-map`: Generates the geographical heat-map data.

4. System Architecture

1. **Client Layer**: Flutter Mobile/Wearable apps collect data via Garmin/HealthKit SDKs.

2. **Communication Layer**: Real-time Telemetry via WebSockets (for live coaching) and REST for sync.

3. **Processing Layer**: Node.js backend calculates physics-based pace adjustments and biomechanical thresholds.

4. **Intelligence Layer**: OpenAI API synthesizes natural language coaching cues based on physics engine triggers.

5. **Data Layer**: PostgreSQL/Prisma stores historical training blocks and high-fidelity telemetry.

3. Phase-by-Phase Antigravity System Prompts

Phase 1: Database Setup & Garmin/HealthKit Sensor Sync

Task: Initialize the PaceFlow backend infrastructure and wearable synchronization layer.

Context: Focus on high-fidelity data ingestion and structured storage.

Instructions:

1. Setup a Node.js Express project with TypeScript and Prisma ORM.

2. Define the PostgreSQL schema in `schema.prisma` including Users, TrainingPlans, Courses, RunSessions, and TelemetrySamples (supporting millisecond-level precision).

3. Implement API webhooks for Garmin Connect and Apple HealthKit to receive asynchronous workout data.

4. Create a telemetry ingestion service that validates and stores incoming sensor data (HR, Cadence, GCT, Vertical Oscillation) from wearable SDKs.

5. Establish a strict review milestone: Ensure the schema supports 100Hz telemetry samples without performance degradation.

Boundary: Do not implement the physics engine or UI in this phase.

Phase 2: Physics-Engine & Elevation-Adjusted Pace Curve Calculator

Task: Develop the core pacing logic based on physical and environmental constraints.

Context: Transform raw GPS/GPX data into an actionable, dynamic racing plan.

Instructions:

1. Build a GPX Parser Service that calculates localized slopes (gradient %) for every 5-meter segment of a course.

2. Implement a Physics Engine that adjusts target pace based on:

- Slope (using standard metabolic cost models for uphill/downhill).

- Weather Data (fetching heat index and wind resistance via external API for the course coordinates).

3. Create an algorithm to generate a "Negative Split" pace curve that compensates for predicted energy expenditure.

4. Expose an endpoint `GET /pacing/curve` that returns a timestamped JSON array of target paces.

5. Review Milestone: Validate the pace curve against a control GPX file with 10% gradients.

Boundary: Focus exclusively on the mathematical model and API; no audio synthesis yet.

Phase 3: Live Bio-Feedback Form Monitor & Audio Coach

Task: Program the real-time coaching logic and audio synthesis.

Context: Convert biomechanical telemetry into live performance corrections.

Instructions:

1. Implement a Signal Processing Service to monitor telemetry thresholds:

- Cadence drops (>5% from baseline).

- Ground Contact Time (GCT) increases indicative of fatigue.

- Vertical Oscillation shifts.

2. Integrate the OpenAI API to generate concise, natural language coaching instructions based on detected form degradation (e.g., "Shorten your stride; you're over-striding on the descent").

3. Implement a Text-to-Speech (TTS) pipeline to stream these instructions to the Flutter client.

4. Establish state tracking for "Coach Interventions" to prevent over-notifying the user.

5. Review Milestone: Verify the latency between a threshold breach and audio cue generation is under 2 seconds.

Boundary: No visualization or simulation logic in this phase.

Phase 4: Monte Carlo Goal Simulator & Fatigue Heat-Map Dashboard

Task: Build the predictive engine and the post-run analytical visualization.

Context: Provide probability-based goal setting and visual training recalibration.

Instructions:

1. Develop a Monte Carlo Simulation Engine in the backend:

    - Run 10,000 iterations per request using variable inputs (fatigue coefficient, weather variance, HR drift).

    - Output a probability distribution of finishing times for a specific GoalTime.

2. Build the Fatigue Heat-Map Logic:

    - Map biomechanical "Form Degradation" scores (calculated from GCT and Cadence) onto GPS coordinates.

    - Export this as a GeoJSON layer for the Flutter Mapbox integration.

3. Finalize the Dashboard API to link fatigue hot-spots directly to training block adjustments (e.g., suggesting more hill work if form breaks on inclines).

4. Review Milestone: Compare simulator confidence intervals against historical run data for accuracy.

Boundary: Finalize all documentation and ensure strict adherence to the defined Tech Stack.